

Flash: Query-Efficient Black-Box Static Malware Evasion through Transferable GAN-Guided Modification Sequences

ANYUAN SANG, Xidian University, China

LI YANG*, Xidian University, China

LU ZHOU, Xidian University, China

JUNBO JIA, Xidian University, China

HUIPENG YANG, Xidian University, China

Machine learning (ML)-based static malware detectors are widely deployed for Portable Executable (PE) files due to their scalability and efficiency, yet they remain vulnerable to carefully crafted adversarial perturbations. Existing black-box evasion methods either rely on transfer attacks, which break down when surrogate and target decision boundaries diverge, or on query-driven searches, which require impractically many queries. We present Flash, a two-phase adversarial framework tailored for static PE malware detection that integrates the strengths of both approaches. In the first phase, a generative adversarial network is trained against heterogeneous surrogate detectors to generate function-preserving PE modifications with inherent evasiveness. In the second phase, an evolutionary optimizer refines these sequences directly against the target model with a dual-objective fitness that balances evasion success and minimal perturbation cost. Experiments on 12,039 VirusShare PE files and six state-of-the-art static detectors demonstrate that Flash reduces query counts by 86% while maintaining bypass rates above 95.8%. Furthermore, adversarial training with Flash-generated samples reduces attack success rates by 82.4%, highlighting Flash's utility for both exposing vulnerabilities and strengthening the robustness of static PE malware detectors.

CCS Concepts: • **Security and privacy** → **Malware and its mitigation**.

Additional Key Words and Phrases: PE file, Evasion Attack, Transferability

ACM Reference Format:

Anyuan Sang, Li Yang, Lu Zhou, Junbo Jia, and Huipeng Yang. 2026. Flash: Query-Efficient Black-Box Static Malware Evasion through Transferable GAN-Guided Modification Sequences. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE037 (July 2026), 22 pages. <https://doi.org/10.1145/3797132>

1 Introduction

Portable Executable (PE) format malware has become a significant cybersecurity threat, causing substantial economic and social damage [5, 48]. As adversarial techniques evolve, malware detection has increasingly relied on machine learning (ML)-based models [40, 55], which have demonstrated remarkable performance in identifying malicious software. However, ML-based detection methods are inherently vulnerable to adversarial perturbations, where even minor modifications to an input sample can lead to incorrect classification [14, 39, 49, 56, 70]. This susceptibility raises concerns

*Li Yang is the corresponding author.

Authors' Contact Information: Anyuan Sang, Xidian University, Xi'an, China, anyuan@stu.xidian.edu.cn; Li Yang, Shaanxi Key Laboratory of Network and System Security, Xidian University, Xi'an, China, yangli@xidian.edu.cn; Lu Zhou, Xidian University, Xi'an, China, zhoulu@xidian.edu.cn; Junbo Jia, Xidian University, Xi'an, China, jbjia@stu.xidian.edu.cn; Huipeng Yang, Xidian University, Xi'an, China, yanghp@stu.xidian.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE037

<https://doi.org/10.1145/3797132>

about the robustness of these detection systems, particularly in static malware detection techniques, necessitating a deeper investigation into adversarial resilience in ML-based malware detection.

Existing adversarial attack techniques against static malware detection can be broadly categorized into white-box and black-box attacks [18, 24, 44, 67, 68]. White-box attacks assume that the adversary has complete knowledge of the target model, including its parameters and gradients, enabling the use of gradient-based methods such as Fast Gradient Sign Method (FGSM) [57] and Padding attacks [35]. However, these attacks rely on strong assumptions that are unrealistic in practical deployment scenarios, where model details are typically inaccessible [44]. Consequently, research has shifted toward black-box attack methodologies, which are more applicable in real-world settings. Black-box attacks can be further divided into transfer-based and query-based approaches [18].

Transfer-based attacks [18, 54, 70] generate adversarial examples using a surrogate white-box model, with the expectation that these samples will remain effective against the target black-box model. While this approach often achieves high attack efficiency, its success is constrained by the architectural differences between the surrogate and target models, which reduce the transferability of adversarial samples (**#C1: Transferability Gap**) [16]. Although various techniques have been proposed to improve transferability [20, 63], they still struggle to consistently succeed across diverse detection models. This limitation is primarily due to the fact that the generator learns modification strategies tailored to the surrogate model rather than the actual black-box model. As a result, if there is a significant discrepancy between the decision boundaries of the surrogate and target models, the initial adversarial samples may exhibit limited evasive capability. Furthermore, most existing transfer-based attacks operate solely in the feature space [31], preventing them from producing valid, executable PE files (**#C2: Feature-Space Infeasibility**).

In contrast, query-based attacks [17, 56, 70] directly interact with the target model, repeatedly querying it and analyzing its hard-label outputs (i.e., malware or benign classification) to optimize the adversarial sample. Query-based methods typically employ evolutionary algorithms, such as genetic algorithms (GA) [8, 17] and reinforcement learning (RL) [21, 30, 38, 43, 56]. Genetic algorithms, for instance, iteratively refine a population of perturbed samples by selecting the most effective adversarial instances for subsequent generations. RL-based approaches define modification actions with associated reward signals, enabling adversarial optimization through reinforcement learning techniques. However, all these methods are inherently iterative and require a large number of queries, which is impractical in real-world attack scenarios due to query rate limitations and detection avoidance mechanisms (**#C3: Query Overhead**).

To address the aforementioned challenges, we propose Flash, a scalable adversarial attack framework that integrates the advantages of both transfer-based and query-based attacks. Flash is designed to efficiently generate adversarial samples while maintaining low query overhead. Specifically, Flash is an end-to-end two-stage framework. In the first stage, it employs a GAN generator trained against multiple heterogeneous surrogate detectors to produce initial perturbation sequences. In the second stage, Flash directly interacts with the target detector and applies a genetic algorithm to iteratively optimize the initially generated samples for query-based evasion.

Flash introduces a set of carefully designed mechanisms that jointly ensure feasibility, transferability, and efficiency in black-box evasion. First, unlike prior attacks that perturb feature vectors and often yield invalid samples, Flash operates directly on the binary level. It leverages twenty-six carefully curated PE-preserving manipulations so that every adversarial sample remains a valid and executable Windows binary (C2: Feature-Space Infeasibility). Second, adversarial examples generated on a single surrogate frequently fail when the target detector's decision boundaries diverge. Flash addresses this issue through a two-fold design: (i) a multi-discriminator GAN trained

against heterogeneous surrogates, including static feature and raw byte detectors, which encourages the generator to capture diverse and generalizable evasion strategies; and (ii) an evolutionary optimization stage that further refines GAN-generated candidates against the target, effectively bridging the residual decision boundary differences (C1: Transferability Gap). Finally, query-driven attacks in the literature typically require hundreds of iterations, making them impractical in real-world scenarios. Flash mitigates this overhead by front-loading evasiveness: the GAN generator produces initial perturbation sequences that already carry strong bypass capability, enabling the evolutionary optimizer to start from near-evasive solutions. This two-stage design dramatically reduces the number of queries needed to achieve high success rates (C3: Query Overhead).

To comprehensively evaluate the performance of Flash, we conduct extensive experiments on a dataset of approximately 12,000 PE-format malware samples from VirusShare [12]. We assess Flash's effectiveness against several state-of-the-art malware detectors, including MalConv [50], MalConvGCT [51], LightGBM [33], Ember [3], ClamAV [11], Avast [4] and VirusTotal [60]. Additionally, we compare Flash against 10 adversarial attack baselines, including GAMMA Padding (GP) [17], GAMMA Section (GS) [17], Partial DOS (PD) [16], Full DOS (FD) [18], Extend [18], Shift [18], Padding [18], MAB [56], Malfox [70] and MalwareTotal [30]. Experimental results demonstrate that Flash outperforms all existing methods in terms of evasion rate, transferability, and generation efficiency, achieving bypass rates ranging from 95.8% to 100% across multiple detectors and reducing query counts by approximately 86% compared to the most competitive baseline.

The key contributions of our work are as follows:

- **Query-efficient Black-box Adversarial Attack.** We present Flash, a systematic adversarial testing framework for static malware detectors. Flash integrates generative modeling and evolutionary optimization to automatically generate functionality-preserving adversarial samples, reducing query cost by up to 86%.
- **High Attack Efficiency.** We implemented a prototype of Flash and conducted extensive experiments, demonstrating that Flash outperforms state-of-the-art methods in terms of attack performance. It achieves over 86% reduction in model queries and an attack success rate exceeding 96.5%. We also validate the transferability of Flash-generated adversarial samples across different detectors.
- **Effective Robustness Enhancement.** We demonstrate Flash's ability to improve the robustness of existing malware detectors. By fine-tuning the detection models with adversarial samples generated by Flash, we observe a reduction in ASR by 82.4%. Moreover, the enhanced models exhibit stronger resilience when confronted with previously unseen black-box detectors.

Ethical Implications & Open Science. Although we propose the FLASH framework from the attacker's perspective, our aim is to help design more resilient and secure systems by understanding and simulating malware evasion, so they can operate reliably in the face of evolving threats. In Section 7.1, we also discuss three defense strategies against Flash, to facilitate further research, we release the data and code used in this study in a replication repository at <https://github.com/anyuan1999/Flash.git>.

2 Related Work

Static machine learning-based malware detection techniques have been widely adopted in commercial anti-virus products and are often regarded as the first line of defense against intrusions [23, 25, 28, 47, 48]. Consequently, evaluating the robustness of such detectors against adversarial examples has become a focal point of recent research. Existing studies on adversarial examples

Table 1. Comparison of representative black-box adversarial attack methods. ✕ to denote not preserve functionality, ⊖ to denote only preserved conceptually but not verification, ✓ to denote preserve functionality

Method	Category	Type	Perturbation	Functionality	Optimizer	Query
MalGAN [31]	Transfer	black-box	feature space	✕	GAN	N/A
Malfox [70]	Transfer	black-box	problem space	⊖	Convolutional-GAN	N/A
GAPGAN [65]	Transfer	black-box	problem space	⊖	GAN	N/A
Shift/Extend.. [18]	Query	black-box	problem space	⊖	GA	High
GAMMA [17]	Query	black-box	problem space	⊖	GA	High
MAB-Malware [56]	Query	black-box	problem space	⊖	RL	High
MalwareTotal [30]	Query	black-box	problem space	✓	PPO	High
Flash (Ours)	Transfer+Query	black-box	problem space	✓	GAN+GA	Low

targeting static malware detectors can be broadly categorized into white-box and black-box attacks (As shown in Table 1).

White-box attacks assume that the adversary has full access to the model’s architecture, parameters, and training data, enabling gradient-based perturbations on input samples to evade detection while preserving malicious functionality. For example, Kolosnjaji et al. [36] proposed the Padding method, which injects gradient-guided bytes into the overlay region of PE files, achieving about a 60% attack success rate on MalConv while maintaining functionality. However, since white-box attacks rely on overly strong assumptions, researchers have shifted attention toward the more realistic black-box threat model.

We mainly focus on problem-space attacks under black-box scenarios. As shown in Table 1, black-box attacks can be further divided into feature-space and problem-space approaches. Feature-space attacks modify feature vectors to produce adversarial representations that evade detectors but cannot yield functional malware samples. For instance, MalGAN proposed by Hu et al. [31] extracts API-call features from PE files and trains a GAN to attack machine learning detectors, but it does not generate actual adversarial malware.

Problem-space attacks, on the other hand, are more practical. GAN-based approaches such as MalFox [70] and GAPGAN [65] operate directly on raw bytes to train GAN models, thereby producing real adversarial malware samples. A key advantage of GAN-based attacks is the transferability of generated adversarial samples: the generator is trained against a surrogate model and can then be used to evade other types of detectors, reducing the need for costly black-box queries.

Another major category is query-based optimization methods, which directly interact with the target detector to optimize adversarial samples. For example, Demetrio et al. [16] introduced the Partial DOS method, perturbing the first 58 bytes of the DOS header to evade detection by exploiting detectors’ reliance on header features. Other techniques further exploit PE structural properties: the Header Fields method [2] modifies section names in the Optional Header to confuse detectors; Demetrio et al. [18] proposed three manipulations: fully rewriting the DOS stub (Full DOS), expanding the header with adversarial content (Extend), and inserting crafted bytes before the first section (Shift). In addition, optimization-based approaches such as GAMMA [17], MAB-Malware [56], and MalwareTotal [30] leverage different algorithms—including genetic algorithms (GA), reinforcement learning (RL), and proximal policy optimization (PPO)—to iteratively refine adversarial malware. However, their drawback is the large number of queries generated during optimization, which is impractical in real-world scenarios since many systems deploy query-monitoring defenses to prevent continuous probing.

In contrast to these existing methods, FLASH combines the strengths of both query-based and transfer-based approaches. It leverages a GAN to train a generator capable of producing perturbation strategies, thereby equipping samples with inherent adversarial properties and substantially

3 Background

PE File Structure. The Portable Executable (PE) format is the standard binary file format used for executable, object code, and DLLs on Windows systems. It provides a well-structured layout that allows the operating system loader to correctly interpret, map, and execute the contents of a binary file. A PE file typically consists of the following components:

DOS Stub: A small legacy program that follows the MZ header. If the executable is run in a DOS environment, it displays a message like “This program cannot be run in DOS mode.”

PE Signature & Headers: Located at the offset specified by `e_lfanew`, the PE signature (PE\0\0) is followed by:

- **File Header:** Contains metadata such as machine type, number of sections, and timestamp.
- **Optional Header (actually mandatory):** Includes crucial information for the loader such as the entry point, image base, alignment values, and a series of **Data Directories**.

Data Directories: A key substructure within the Optional Header. Each entry in this array provides a pointer (Relative Virtual Address, RVA) and size of various tables, such as the **Import Table** and **Import Address Table (IAT)**. These point to dynamic linking data used during runtime.

Section Table (Section Headers): Immediately follows the PE headers. Each section header describes the characteristics of a section, including its name, virtual size, raw size, RVA, and attributes (e.g., code/data, read/write/execute permissions).

Sections: These are the actual data blocks in the file. Each section stores specific content based on its type:

- `.text`: Contains the executable instructions.
- `.data`: Stores initialized variables.
- `.rsrc`: Holds resources like dialogs, icons, and version info.
- `.idata`: Contains import-related information, including IAT.
- `.reloc`: Stores relocation information if the image is not loaded at its preferred base address.

Import Table and IAT: Using entries in the Data Directory, the loader accesses the Import Table to identify required external libraries and functions. The **IAT** (Import Address Table) is then populated with the actual memory addresses of these functions at runtime. As illustrated in the figure, each IAT entry links to a resolved function address, enabling dynamic linking.

These components together define the logical structure of a PE file, supporting its correct loading and execution within the Windows runtime environment.

4 Threat Model

In this work, we focus on generating adversarial Portable Executable (PE) malware examples under the **black-box threat model in problem space**, where the adversary has no access to the internal

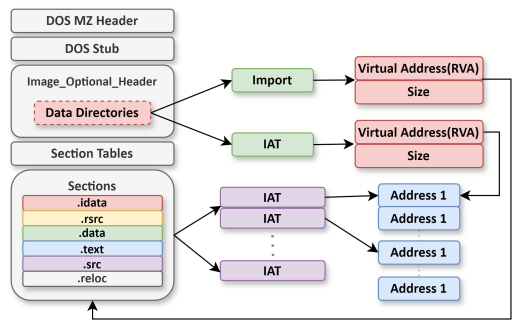


Fig. 1. An example PE file structure in [70].

parameters, architecture, or gradients of the target static detector. The adversary can only interact with the detector through prediction outputs, either class labels (hard labels) or confidence scores (soft labels). We assume the attacker is allowed a limited number of queries to the detector, which is a standard assumption in black-box adversarial attack studies [30, 53, 59]. This is realistic since many cloud-based malware scanning services provide APIs with query quotas, and attackers may leverage distributed infrastructures (e.g., varying IPs/accounts) to bypass such restrictions.

Let $f : \mathcal{X} \rightarrow \mathcal{Y}$ denote a black-box malware detection function, where $\mathcal{X}$ is the space of malware binaries (e.g., PE files), and $\mathcal{Y} = \{0, 1\}$ is the set of prediction labels, where 1 denotes *malicious* and 0 denotes *benign*. Given a known malicious input $x \in \mathcal{X}$ such that $f(x) = 1$, the goal of the adversary is to find a perturbed sample $x^* \in \mathcal{X}$ that satisfies the following constraints:

$$f(x^*) = 0, \quad (1)$$

$$\mathcal{F}(x, x^*) = 1, \quad (2)$$

$$\text{Queries}(f, x^*) \leq B \quad (3)$$

Here, $\mathcal{F}(x, x^*) = 1$ indicates that the functional semantics of the original malware are preserved in the modified version x^* , and B is the upper bound on the number of model queries allowed during the attack.

To model the perturbation process in the problem space, we define a finite set of functionality-preserving modification operations:

$$\mathcal{O} = \{o_1, o_2, \dots, o_k\}$$

Each operation $o_i \in \mathcal{O}$ transforms the input binary x into a new binary x' , such that $x' = o_i(x)$ and $\mathcal{F}(x, x') = 1$. A complete adversarial example is obtained by applying a sequence of operations:

$$x^* = \mathcal{T}(x; \mathbf{o}_{1:t}) = o_t \circ \dots \circ o_2 \circ o_1(x)$$

The objective is to find the optimal operation sequence $\mathbf{o}_{1:t}$ that minimizes the evasion cost under the constraints defined above. This can be framed as a constrained discrete optimization problem:

$$\min_{\mathbf{o}_{1:t} \in \mathcal{O}} \mathcal{L}(f(x^*)) + \lambda \cdot \text{Cost}(x, x^*),$$

where $\mathcal{L}(f(x^*))$ is the detector loss (e.g., predicted confidence for maliciousness), and $\text{Cost}(x, x^*)$ measures the extent of the modifications (e.g., number of operations, byte changes). The scalar λ controls the trade-off between evasion success and modification cost.

5 Methodology

5.1 Overall Framework

Flash adopts a two-stage adversarial attack pipeline, as illustrated in Figure 2, which efficiently generates evasive and function-preserving adversarial malware samples while minimizing query costs to the target detector. The framework seamlessly integrates generative modeling and evolutionary search across its two stages.

First, following previous research [16–18, 30, 56, 62], we define a set of discrete PE file modification operations—including Section operations, Header modifications, instruction manipulations, and Overlay actions—which have been shown to preserve malware functionality. This provides a robust foundation for the subsequent training and attack phases.

In the first stage (generator training), the framework employs a generator network that maps random latent variables (using differentiable techniques) to sequences of discrete modification

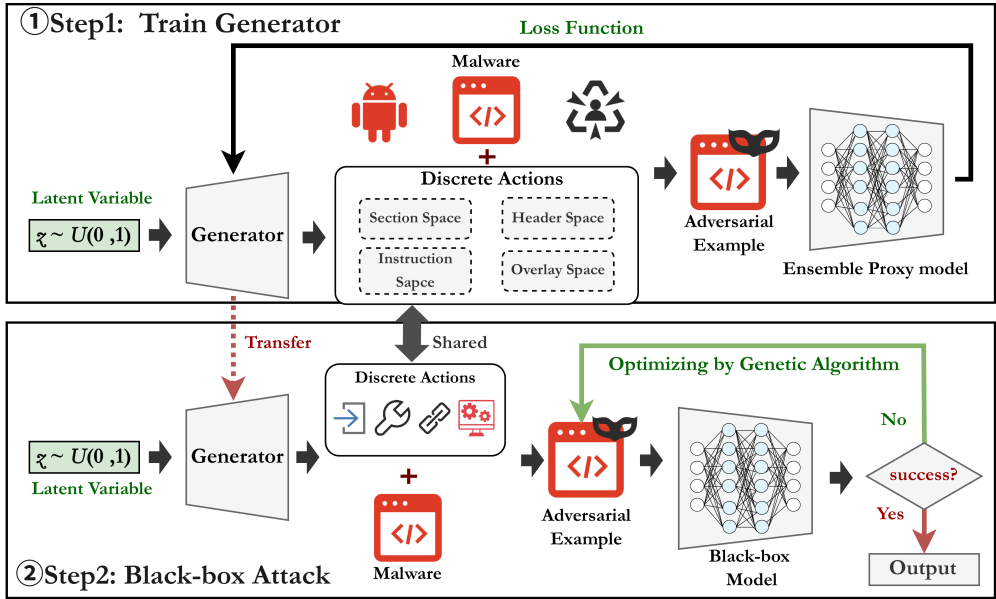


Fig. 2. Framework of Flash. Flash first trains a generator to map latent vectors to valid PE modification strategies using surrogate detector feedback. These strategies initialize a genetic algorithm that refines them through mutation and selection, aiming to maximize evasion success while preserving functionality under black-box query constraints.

actions. These actions cover functional-equivalent modifications to PE sections, overlays, import tables, and more. All available actions are selected from a predefined set to guarantee that each adversarial modification does not affect the original functionality of the malware sample. Each candidate action sequence is applied to the malware sample to generate an adversarial example, which is subsequently evaluated by a surrogate black-box model composed of multiple detectors. The generator is trained using a loss function based on the detection scores of these surrogate detectors, thus learning to produce modification strategies that are effective in evading the target detection model.

In the second stage (attack), the trained generator is used to rapidly generate candidate modification strategies for previously unseen malware samples. These strategies are provided as the initial population for a genetic algorithm, which iteratively refines them to further improve attack success rates against the black-box target detector. At each iteration, candidate action sequences are applied to the malware sample, and the resulting adversarial examples are evaluated via queries to the actual black-box detector. The genetic algorithm selects, recombines, and mutates these strategies according to a multi-objective fitness function that rewards both successful evasion and minimal perturbation. This optimization process continues until an adversarial example is produced that successfully bypasses the black-box detector or the maximum allowed query budget is exhausted.

5.2 Functionality-Preserving Modification

To ensure that all adversarial modifications preserve the original functionality of malware samples, we systematically design and implement a comprehensive set of discrete PE file manipulation strategies. These actions are carefully selected to avoid breaking binary structure or execution

capability, and are grounded in both state-of-the-art and novel manipulation techniques documented in prior research.

As summarized in Table 2, our operation set covers four main categories: header manipulation (e.g., zeroing certificates, modifying timestamps, and adjusting linker versions), section manipulation (e.g., adding new sections, injecting benign content, and renaming sections), overlay manipulation (e.g., appending random or benign data to the overlay), and compound manipulations that ensemble multiple basic actions or apply code-level obfuscations. Each modification is implemented using robust binary rewriting tools and is validated to ensure functional correctness through automated parsing and sandbox execution.

This operation set largely follows a range of techniques proposed in previous work [30], functionality-preserving action space for both GAN-based strategy learning and subsequent evolutionary optimization.

5.3 GAN-Based Modification Strategy Learning

Flash models the task of generating functionality-preserving adversarial malware as a problem of learning discrete modification strategies. The generator network is trained to output a sequence of semantic PE manipulation actions, each chosen from a well-defined operation set (see Table 2), with the guarantee that no modification impairs the malware's original functionality. Each action corresponds to a meaningful binary operation, such as changing header metadata, injecting benign section content, or extending the DOS stub.

During training, a random latent variable z is sampled from a uniform distribution and fed into the generator G_θ . The generator produces logits for a sequence of K actions, where each action is sampled as a categorical variable using the Gumbel-Softmax trick [32]:

$$a_i = \text{one_hot} \left(\arg \max_j \text{GumbelSoftmax}(o_{i,j}, \tau) \right),$$

where $o_{i,j}$ are the generator outputs for the i -th position and τ is the temperature parameter annealed during training.

Algorithm 1: GAN-based Modification Strategy Learning with Ensemble Surrogate Detectors

Input: Malware sample x ; action set $\mathcal{O}$; ensemble of surrogate detectors $\{f_1, f_2, \dots, f_M\}$; generator G_θ ; training steps N

```

1 for  $t = 1$  to  $N$  do
2   Sample latent variable  $z \sim \mathcal{U}(0, 1)$ ;
3   Generate action logits  $O = G_\theta(z)$ ;
4   For each  $i = 1$  to  $K$ , sample action  $a_i$  using Gumbel-Softmax;
5   Apply  $A = [a_1, \dots, a_K]$  to  $x$  to obtain  $T_A(x)$ ;
6   Compute ensemble loss:  $\mathcal{L}_G = \frac{1}{M} \sum_{m=1}^M f_m(T_A(x))$ ;
7   Update  $\theta$  by descending  $\nabla_\theta \mathcal{L}_G$ ;
8   Anneal  $\tau$  according to schedule;
```

The sampled action sequence $A = [a_1, \dots, a_K]$ is then applied to an original malware sample x , yielding a perturbed sample $T_A(x)$. Instead of relying on a single surrogate model, Flash leverages an ensemble of ML-based surrogate detectors $\{f_1, f_2, \dots, f_M\}$, such as MalConv and Ember, to evaluate the adversarial example. The generator is optimized to minimize the averaged maliciousness

Table 2. Implemented novel and state-of-the-art manipulations within Flash, refer from [30]

Category	No.	Manipulation	Description	Refs
Header Manipulation	1	Extend the DOS Stub	Extend the DOS Stub by one file_alignment (at large 6*file_alignment) and inject random bytes.	[16, 18, 35]
	2	Zero CheckSum	Set the member CheckSum to zero.	[18, 38]
	3	Change TimeDateStamp	Change the member TimeDateStamp in File Header to a int value by random selection that can be zero or any datetime after 1970-1-1.	[62]
	4	Modify IMAGE_DIRECTORY_ENTRY_DEBUG	Set the IMAGE_DIRECTORY_ENTRY_DEBUG member to zero.	[1, 2]
	5	Modify Linker/OS Versions	Modify the MajorLinkerVersion, MinorLinkerVersion, MajorOperatingSystemVersion, MinorOperatingSystemVersion, MajorImageVersion, MinorImageVersion respectively.	[30]
	6	Zero Optional Header Certificate	Zero out the certification in the Optional Header.	[38, 56]
Section Manipulation	7	Rename a section	Rename a section by randomly selecting a section name extracted from benign binaries.	[2, 18]
	8	Import unused library	Import a library that has never been called. Counterfeit the IAT and INT in the Data Directory.	[17, 31]
	9	Copy/Move IAT/INT	Copy the original Import Address Table (IAT) and Import Name Table (INT) to a position at the end of the final section, but before the overlay.	[30]
	10	Inject benign content (new section)	Create a section and inject content extracted from benign binaries' sections.	[1]
	11	Inject benign strings (new section)	Create a section and inject content, the injected content is extracted from the benign binaries' strings.	[56, 62]
	12	Inject bytes in section cave	Inject bytes in unused space in a section. Bytes injected are extracted from benign PE's sections. centering	[17, 66]
	13	Inject before first section	Inject benign content before the first section.	[18]
Overlay Manipulation	14	Append random bytes	Append random bytes to the input overlay.	[1, 38]
	15	Append benign content (overlay)	Append benign content extracted from a random benign binary's section to input malware overlay.	[17, 37]
	16	Append benign hex data (overlay)	Append benign hexadecimal data extracted from a random benign binary to input malware overlay.	[2, 66]
	17	Append benign ASCII string	Append ASCII code converted from strings extracted from a random benign binary.	[26]
Compound Manipulation	18	Change all modifiable bytes	Change all modifiable bytes in DOS Header, DOS Stub, Optional Header, and Section Table to random bytes.	[35]
	19	Alter DOS Header (58 bytes)	Alter 58 modifiable bytes in DOS Header to random bytes.	[16]
	20	Alter DOS Header and Stub	Alter modifiable bytes in DOS Header and DOS Stub to random bytes.	[18]
	21	Ensemble header manipulations	Conduct ensemble header manipulations (i.e. manipulations 1 to 5).	[30]
	22	Ensemble section manipulations	Conduct ensemble sectional manipulations (i.e. manipulations 6 to 12).	[30]
	23	Compression/encryption	Compress and encrypt the input, also refers as instruction reordering or code randomization.	[46, 61, 62]

probability predicted by the ensemble, with the following loss:

$$\mathcal{L}_G = \frac{1}{M} \sum_{m=1}^M f_m(T_A(x)).$$

The overall training process alternates between sampling z , generating candidate modifications, evaluating with the ensemble surrogate models, and updating the generator parameters with gradients from $\mathcal{L}_G$. The Gumbel-Softmax temperature τ is gradually reduced to encourage more deterministic and realistic action choices.

Algorithm 1 summarizes the core training loop: the generator efficiently proposes modification strategies that are robust against multiple surrogate detectors, providing high-quality candidates for subsequent evolutionary optimization.

5.4 Black-Box Attack

Although the GAN-based generator can quickly propose modification strategies that may evade surrogate detectors, discrepancies between real-world black-box detectors and surrogate models may render these strategies ineffective. To bridge this gap, Flash refines the generator's candidate strategies through an evolutionary optimization process, thereby enabling precise adaptation to the actual black-box model while effectively controlling both query cost and perturbation cost. The evolutionary phase begins by sampling a set of candidate modification strategies from the trained generator, each represented as a sequence of discrete actions. These candidates are applied to the original malware sample, yielding an initial population of adversarial examples. For each individual, functionality is verified and evasiveness is assessed by querying the true black-box detector.

Central to this phase is a multi-objective fitness function designed to reward both successful evasion and minimal, stealthy modification. Specifically, for a strategy A , the fitness is computed as:

$$\text{Fitness}(A) = \alpha \cdot [1 - f_{\text{bb}}(T_A(x))] + \beta \cdot \mathcal{S}(A)$$

where f_{bb} denotes the black-box detector score, $\mathcal{S}(A)$ penalizes excessive modifications (e.g., via the negative ℓ_0 -norm of the action sequence), and α, β are hyperparameters controlling the trade-off.

At each evolutionary iteration, parents are selected via tournament selection, favoring strategies with higher fitness. New candidate strategies are generated through crossover and mutation, producing diverse offspring while inheriting effective traits. All new adversarial samples are validated for functionality before being evaluated by the detector. Elite strategies from the previous generation are preserved to ensure consistent progress.

Efficiency is further enhanced by two mechanisms. First, early termination is triggered as soon as a strategy is found that successfully evades the detector, preventing unnecessary queries. Second, feature caching stores effective modification patterns observed during evolution, enabling their reuse and accelerating convergence.

The evolutionary loop continues until either an evasive adversarial sample is produced or the query budget is exhausted. The best-performing modification sequence is output as the final attack strategy, balancing evasiveness, stealth, and query efficiency.

The Algorithm 2 summarizes the core evolutionary optimization. The evolutionary refinement ensures that the generated adversarial malware is not only effective against the specific black-box detector but also minimally perturbed and query-efficient, providing robust real-world evasion capability.

6 Experimental Evaluation

In this section, we address the following key research questions to comprehensively evaluate the effectiveness and practicality of Flash in real-world adversarial scenarios:

- **RQ1:** How effective is Flash compared to existing methods in evading SOTA malware detectors?
- **RQ2:** What is the contribution of each core component and perturbation strategy within Flash?
- **RQ3:** How can the optimal configuration parameters for Flash be selected?
- **RQ4:** How transferable are the attack strategies generated by Flash, and can Flash be used to improve the robustness of malware detectors?

Algorithm 2: Evolutionary Optimization for Black-box Evasion

Input: Malware sample x ; generator G_θ ; action set $\mathcal{O}$; black-box detector f_{bb} ; population size P ; max generations G

```

1 Initialize population  $\mathcal{P}_0$  by sampling  $P$  strategies from  $G_\theta$ ;
2 for  $g = 1$  to  $G$  do
3   For each  $A$  in  $\mathcal{P}_{g-1}$ : apply  $A$  to  $x$ , check functionality, and query  $f_{bb}$  for score;
4   Compute fitness for all individuals;
5   if any  $A$  achieves  $f_{bb}(T_A(x)) < \tau$  then
6     break // Early stopping if evasion found
7   Select parents via tournament selection;
8   Generate offspring by crossover and mutation;
9   Form new population  $\mathcal{P}_g$  with elite preservation;
10  Update feature cache with successful strategies;
11 Return best strategy found;
```

6.1 Experimental Setup

In this section, we present the datasets, target detection models, and baseline methods used in our experiments, as well as the implementation details of Flash and the experimental environment.

Detectors. To comprehensively evaluate the evasion capability of Flash against different detectors, we select three academic malware detectors, namely Ember [3], MalConvGCT [51], and LightGBM [33], as well as three commercial detectors, ClamAV [11], Avast [4], and VirusTotal [60]. Since academic detectors can provide soft-label outputs, we perform soft-label attacks against the three academic detectors, while for ClamAV and Avast, which generally only provide hard-label outputs, we adopt hard-label attacks. As VirusTotal’s query interface is restricted and its output represents an aggregated vote from multiple engines, it is not suitable for query count or evasion rate comparisons. Instead, we use VirusTotal to evaluate the transferability of Flash (In RQ4).

Table 3. Summary of malware detectors used for evaluation.

Detector	Type	Features
Ember [3]	Academic (ML)	Ember2018 features
MalConvGCT [51]	Academic (DL)	Raw bytes
LightGBM [33]	Academic (ML)	Ember2024 features
ClamAV [11]	Commercial	Signature + heuristic
Avast [4]	Commercial	Proprietary staticdynamic
VirusTotal [60]	Commercial	Aggregated multiple engines

Ember [3]: Ember is a widely used machine learning-based malware detector that leverages gradient-boosted decision trees (GBDT) trained on static features extracted from Ember2018 dataset. It is recognized for its high detection accuracy and is a standard academic benchmark in adversarial malware research.

MalconvGCT [51]: Based on the MalConv architecture [50], MalConvGCT introduces the Global Channel Gating (GCT) mechanism, which enables the model to efficiently capture feature interactions across the entire input space while significantly improving training efficiency and memory usage. Through a constant-memory convolutional design, it can process ultra-long input sequences (e.g., millions of bytes), thereby extending the applicability of MalConv to large-scale PE files

Table 4. Summary of baseline functionality-preserving adversarial malware generation methods.

Methods	Attack Category	Attack Type	Perturbation Domain	Optimize
Malfox [70]	Transfer	Black-box	Problem space	Convolutional-GAN
GP [17]	Query	Black-box	Problem space	Evolutionary (GA)
GS [17]				Evolutionary (GA)
PD [16]				Evolutionary (GA)
FD [18]				Evolutionary (GA)
Extend [18]				Evolutionary (GA)
Shift [18]				Evolutionary (GA)
Padding [18]				Evolutionary (GA)
MAB [56]				Evolutionary (RL)
MT [30]				Evolutionary (PPO)

LightGBM [33]: In the EMBER2024 benchmark, LightGBM is employed as a primary baseline model for malware detection. The model achieves outstanding results, with a ROC AUC of 0.9949 on the EMBER2024 test set and a true positive rate (TPR) of 94.48% at 1% false positive rate (FPR).

ClamAV [11]: ClamAV is an open-source commercial antivirus engine that relies on a combination of signature-based and heuristic detection techniques. It is widely deployed in real-world security products and serves as a practical benchmark for evaluating adversarial evasion in commercial settings.

VirusTotal [60]: VirusTotal is a cloud-based commercial malware detection service that aggregates multiple detection engines, including both signature-based and machine learning models. It offers comprehensive coverage and high detection reliability, making it an important target for evaluating the real-world impact and transferability of adversarial malware attacks.

Datasets. Since some popular datasets, such as the MS Big15 [34], EMBER [3], and SOREL-20M [29], only provide pre-extracted feature data rather than the original PE files, they are not suitable for evaluating the functional usability of generated adversarial samples. Following prior work [17, 30, 56] on functionality-preserving adversarial malware generation, we use samples from the VirusShare and VirusTotal Academic Share datasets, both of which provide access to complete raw PE files. We collect approximately 12,039 PE samples in total and select 9,000 samples that are consistently recognized as malware by our target detectors.

Baselines. To comprehensively compare the performance of Flash with state-of-the-art approaches, we select nine widely used **functionality-preserving black-box** adversarial malware generation methods, including GAMMA Padding (GP) [17], GAMMA Section (GS) [17], Partial DOS (PD) [16], Full DOS (FD) [18], Extend [18], Shift [18], Padding [18], Malfox [70], MAB [56], and MalwareTotal [30].

Metrics. We employ multiple metrics to comprehensively evaluate the performance of FLASH, including the attack success rate (ASR) against detectors, query cost, perturbation cost, time overhead, and functionality preservation.

The **attack success rate (ASR)** is defined as:

$$ASR = \frac{\# \text{ successfully evaded samples}}{\# \text{ total samples}}.$$

Query denotes the number of requests made to the black-box detector. **Perturbation cost** is measured as the ratio of the added bytes to the original file size, reflecting how much additional content is injected during the perturbation process. **Time overhead** refers to the average processing time required per sample. **Functionality preservation** is validated through a two-stage process: (i) structural integrity is checked using the Pefile library to ensure that perturbed PE files remain

well-formed, and (ii) executability is verified by running the generated adversarial samples in the Cuckoo sandbox.

Implementation. We implement Flash based on the secml-malware framework [15], with PE file manipulations performed using the Pefile library [7]. For functionality-preserving perturbations, we adopt the implementations from MalwareTotal [30]. All experiments are conducted on a personal computer equipped with a Ryzen R7 5700x 4.3GHz CPU and 64GB of RAM.

We use 7,000 PE files, with MalConv and Ember serving as surrogate models to train the GAN module. Subsequently, we conduct black-box attacks on 2,000 malware samples. For comparison, we select ten representative attack methods as baselines. The baseline implementations are based on the open-source attack library ToucanStrike [58]; the implementation of MalwareTotal [30] is provided directly by the authors; and MAB [56] is evaluated using its original codebase. Since the Malfox [70] method has no open-source implementation, we reproduce it according to the settings described in its paper. We set a query limit of 50 and configure Flash with an initial perturbation sequence length of 5 (the rationale behind these parameter settings will be further discussed in RQ3). All methods are evaluated on the same set of PE files, and we systematically record the number of model queries, perturbation cost, attack success rate, and execution time.

6.2 RQ1: Performance of Flash

In this section, we evaluate the performance of Flash and compare it with existing attack methods. First, we examine Flash against three representative and widely used academic detectors Ember, MalConvGCT, and LightGBM under soft-label attacks, and compare it with ten state-of-the-art baseline methods. Next, we further investigate the performance of Flash in hard-label attacks against two commercial consumer-grade antivirus solutions.

Table 5. Effectiveness of Flash and baselines against several academic detectors in soft label scenario (**bold** is first, underline is second).

Methods	MalconvGCT				Ember				LightGBM				Functionality
	ASR(%)	Querys	Cost	Time(h)	ASR(%)	Querys	Cost	Time(h)	ASR(%)	Querys	Cost	Time(h)	
FD	46.1	44622	1.72	6.92	3.0	44582	6.00	10.92	5.6	45121	3.20	9.85	96%
PD	42.3	39533	<u>1.21</u>	8.18	2.0	41332	3.10	11.05	3.2	40144	2.60	10.15	34.1%
Extend	65.0	34637	2.20	10.63	13.0	44253	1.30	14.27	20.5	43612	1.50	12.86	63%
Shift	29.3	43008	2.00	8.30	18.1	39113	<u>1.00</u>	16.12	21.7	40227	1.10	15.04	97%
Padding	33.6	45360	1.80	9.68	37.6	40694	<u>1.00</u>	12.13	35.8	41987	1.20	11.26	100%
GS	85.8	44528	1.60	14.32	79.0	41137	1.30	24.25	81.4	43256	1.40	21.36	96%
GP	82.5	43226	1.50	12.25	67.0	43778	<u>1.00</u>	29.45	73.5	42891	1.20	25.10	96%
MAB	78.5	24621	4.10	5.95	76.4	48231	1.10	5.63	74.8	46512	<u>1.00</u>	6.12	95%
Malfox	73	N/A	1.41	0.97	67.5	N/A	1.33	0.97	61.3	N/A	1.21	0.97	74%
MT	<u>95.6</u>	<u>21834</u>	0.43	2.35	<u>93.1</u>	<u>23711</u>	0.91	2.55	<u>91.3</u>	<u>22341</u>	0.88	2.40	100%
Flash	100	3216	1.65	<u>1.55</u>	96.5	2813	1.85	<u>2.02</u>	95.8	2981	1.72	<u>1.88</u>	100%

For Academic Detectors (Soft-Label Attacks). The experimental results are presented in Table 5. We observe that Flash achieves the highest attack success rates across all three detectors, reaching 100% on MalConvGCT, 96.5% on Ember, and 95.8% on LightGBM. In addition, Flash demonstrates superior query efficiency (reducing query counts by 86%) and achieves an execution time that is second only to transfer-based methods, while being faster than all query-based approaches. We attribute this advantage to Flash’s robust generator, which provides strong initialization for perturbation sequences, thereby reducing the required optimization iterations and significantly accelerating the algorithm. It is worth noting that Flash does not achieve the lowest perturbation cost. This is mainly because the initial perturbation sequence already incorporates several modification actions, leading to slightly higher perturbation compared to stepwise optimization algorithms. Moreover,

we verify the functionality of PE files through two complementary checks: (1) ensuring that the file format remains intact, and (2) executing each binary in a sandbox environment. Flash successfully preserves the functionality of all PE samples.

Table 6. Effectiveness of Flash and baseline methods against ClamAV and Avast in hard label scenario (**bold** is first, underline is second).

Methods	ClamAV				Avast				Functionality
	ASR(%)	Querys	Cost	Time (h)	ASR(%)	Querys	Cost	Time (h)	
MAB	44.1	11088	2.2	4.88	29	13236	1.9	6.08	94%
Gamma Sections	24.7	14200	2	7.8	11.5	15960	2.9	9.08	96%
Malfox	43	N/A	1.6	1.08	37.5	N/A	1.46	1.08	75%
MalwareTotal	<u>91.1</u>	5732	0.9	3.08	<u>92.7</u>	6992	1	3.88	100%
Flash	99	2332	<u>1.4</u>	1.52	97.3	2216	<u>1.3</u>	1.72	100%

For Commercial Antivirus (Hard-Label Attacks). In addition to the soft-label attacks against the three academic detectors, we further evaluate the effectiveness of Flash on two consumer-grade commercial antivirus products. Since commercial AVs typically provide only binary classification results, we adapt the fitness function in the optimization process to accommodate hard-label scenarios. For comparison, we select the best-performing baselines from Table 5, namely MAB, MalwareTotal, GAMMA Sections, and Malfox.

The results, shown in Table 6, indicate that Flash significantly outperforms all baselines in terms of attack success rate (ASR), query efficiency, and execution time on both ClamAV and Avast. Specifically, Flash achieves an ASR of 99% on ClamAV and 97.3% on Avast, with only 2,332 and 2,216 average queries, respectively, and an execution time second only to the transfer-based Malfox method (1.52 hours and 1.72 hours). In contrast, MAB and GAMMA Sections require more than three to six times as many queries and considerably longer execution times, yet yield much lower ASRs (44.1% and 24.7% on ClamAV, 29% and 11.5% on Avast). Although the transfer-based Malfox method achieves the shortest execution time, its ASR is very low. MalwareTotal performs relatively well but still falls behind Flash in both success rate and query efficiency. These results demonstrate that Flash not only achieves state-of-the-art performance on academic detectors but also exhibits high effectiveness against commercial antivirus systems, even under strict query limitations and functionality-preserving constraints.

Answer to RQ1: By combining the advantages of query-based and transfer-based attacks, Flash achieves the highest ASR against both academic and commercial detectors, while also attaining the lowest query counts and the most efficient execution time compared to existing state-of-the-art methods.

6.3 RQ2: Ablation Study

In this section, we conduct a series of ablation studies to investigate the contribution of each component in Flash. First, to separately evaluate the effectiveness of the GAN and GA algorithms, we design two variants: Flash w/o GAN and Flash w/o Evo, which use only the GAN or only the Evo algorithm to optimize the modification operation sequences for PE files, respectively. Next, to validate the effectiveness of the evolutionary optimization algorithm, we employ two baseline optimization methods, namely PPO and Random, to optimize the operation sequences.

We first construct the Flash w/o GAN variant by removing the GAN module from Flash. In this model, the initial perturbation sequences are generated randomly, and GA is subsequently used for

Table 7. Ablation Experiments on MalConv, Ember, and LightGBM

Method	MalConvGCT				Ember				LightGBM			
	ASR	Querys	Cost	Time (h)	ASR	Querys	Cost	Time (h)	ASR	Querys	Cost	Time (h)
Flash w/o GAN	87%	34192	1.5	3.20	84.6%	39200	1.67	4.48	83%	42720	<u>1.53</u>	5.28
Flash w/o Evo	88%	N/A	<u>1.6</u>	0.72	<u>84.4%</u>	N/A	<u>1.6</u>	0.72	81.4%	N/A	1.6	0.72
PPO	81%	15088	0.73	6.88	77.8%	15200	0.89	6.92	81.3%	17240	1.03	8.16
Random Search	56%	53664	4.1	10.60	57%	54000	4.1	10.92	52%	55652	4.1	11.24
Flash	100%	<u>3216</u>	1.65	<u>1.55</u>	96.5%	<u>2813</u>	1.85	<u>2.02</u>	95.8%	<u>2981</u>	1.72	<u>1.88</u>

optimization. Next, we construct the Flash w/o GA variant by removing the GA module; this model directly utilizes the GAN-trained generator to produce perturbation sequences, which are then applied in black-box attacks. The PPO-based method employs Proximal Policy Optimization to learn action sequences that maximize evasion, while the random search method directly generates random modification sequences to alter the PE files during the optimization process.

The results are presented in Table 7. The experimental results clearly demonstrate that Flash achieves the highest attack success rate (ASR) with the lowest average number of queries. We observe that the GAN optimization module is capable of generating strong initial perturbation sequences, achieving an ASR of 88%. This indicates that the primary function of the GAN module in Flash is to produce high-quality initial samples, which substantially reduces both the number of optimization queries and the overall time required by the subsequent GA. The GA module, on the other hand, is able to address the discrepancy between the initial samples and the decision boundary of the target detector, thereby further improving the ASR. In contrast, the PPO-based method exhibits inferior performance, primarily due to its inefficiency in exploring the high-dimensional, sparse-reward search space and the substantial computational overhead associated with training reinforcement learning agents in this domain. Nevertheless, all advanced methods significantly outperform random perturbation, highlighting the necessity of intelligent initialization and optimization strategies for effective adversarial malware generation.

Answer to RQ2: By performing ablation studies on Flash’s GA and GAN components and comparing with alternative optimization algorithms, we find that the GAN module in Flash primarily serves to generate high-quality initial samples, while the GA module helps bridge the gap between the surrogate model and the actual target model’s decision boundary, thereby improving the ASR.

6.4 RQ3: Hyper-parameter Sensitive Analysis

In this section, we investigate the sensitivity of Flash to different component and parameter choices. Specifically, we examine several key parameters that are likely to affect Flash’s performance, including the query limit q , the length l of the operation sequence, the noise dimension n of the GAN. S

Query Limit q . Query is crucial for black-box attacks, as fewer queries make the attack more practical in real-world settings. To evaluate how Flash performs under different query limits, we tested it with increasing query budgets and compared it with three strong baselines: MalwareTotal, GAMMA Sections, and MAB. Figures Figure 3.1, Figure 3.2, and Figure 3.3 show the results on the MalConvGCT, Ember, and LightGBM detectors. We observed that the attack success rates of all methods improve with more queries, peaking around 200 queries. However, Flash achieves its best

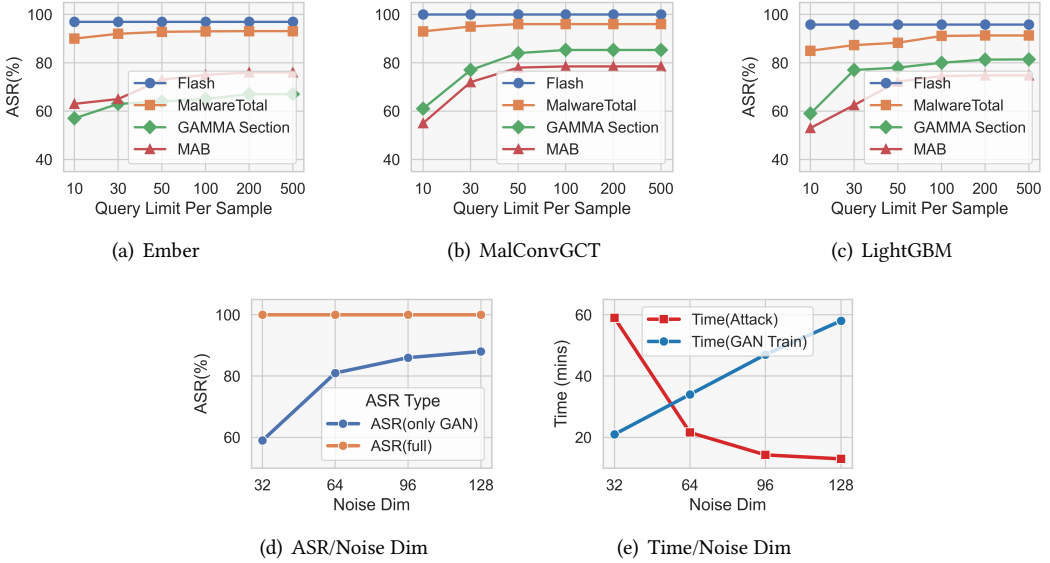


Fig. 3. Impact of different Query-Limit and noise dimensions of GAN.

performance with fewer than 10 queries. While MalwareTotal performs better than the others due to its rich set of modifications, Flash still surpasses it by reaching higher success rates with much fewer queries.

Noise Dimension n . To investigate the impact of noise dimensionality on Flash’s generation capability, we conducted ablation studies with dimensions set to 32, 64, 96, and 128. As shown in Figure 3.(3) and Figure 3.(4), increasing the noise dimension significantly improves the quality of initially generated adversarial strategies (ASR(only GAN)), rising from 59% to 88%. After evolutionary refinement, however, the final attack success rate (ASR(full)) remains at 100% across all settings. Notably, a higher noise dimension reduces the time cost in the attack phase, indicating faster convergence and improved generator efficiency. On the other hand, training the GAN becomes more time-consuming with larger dimensions. Therefore, selecting an appropriate noise dimension is essential to balance training overhead and attack performance. In our experiments, 64-dimensional noise achieves a favorable trade-off.

Operation Sequence Length l . The operation sequence length l in Flash determines the number of perturbations applied. Shorter l may lack sufficient modification space, while longer ones increase GAN training complexity and risk excessive changes that reduce the ASR. To evaluate this trade-off, we experimented with l ranging from 2 to 8. As shown in Fig. 4, the ASR increases with longer sequence lengths and reaches its peak at length 5 (subfigure a), while perturbation overhead (up to 2.18 $\times$, subfigure c) and time cost (up to 2.5 $\times$, subfigure b) continue to rise. The experimental

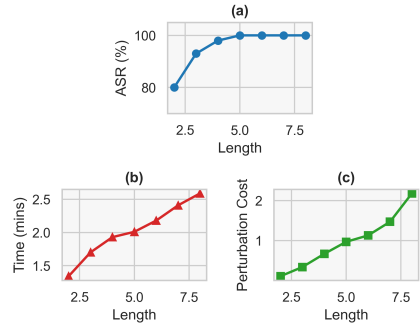


Fig. 4. Impact of different lengths of operation sequence set.

results indicate that ASR, perturbation cost, and time cost all increase as the sequence length grows. Therefore, a moderate sequence length of 5 is recommended to achieve the highest ASR while maintaining a balance between effectiveness and efficiency.

Answer to RQ3: We investigated the impact of different hyperparameter settings on Flash's performance, including the query limit, operation sequence length, and noise dimension. We found that the optimal parameters for achieving high performance while maintaining low time overhead are: query limit $q = 10$, sequence length $l = 5$, and noise dimension $n = 64$.

6.5 RQ4: Transferability and Robustness

In this section, we first investigated the transferability of Flash's adversarial attacks. Next, we explored how to enhance existing detectors using Flash to improve their robustness.

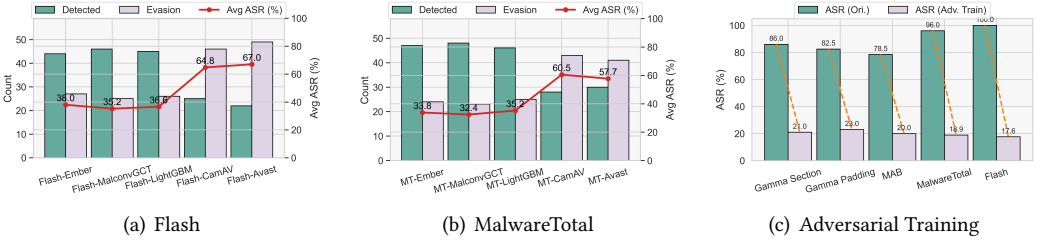


Fig. 5. Comparison of evasion performance between Flash and MalwareTotal across multiple detectors.

Transferability of Flash. The transferability of adversarial attacks arises from the inherent flaws of machine learning models [19], similar to the field of computer vision where adversarial samples often transfer across models. Since many commercial antivirus (AV) engines integrate ML-based detectors, it is essential to evaluate the transferability of Flash-generated attacks. We used Flash to generate 2,000 adversarial samples against different detectors, including Ember, MalConvGCT, LightGBM, ClamAV, and Avast, and scanned them with VirusTotal (covering a total of 71 AV engines). We then recorded the number of AVs bypassed and the number of AVs that detected the samples. For comparison, we adopted MalwareTotal as the second most effective baseline. As shown in Figure 5, attacks against pure ML models exhibited lower transferability, while those against commercial AVs demonstrated stronger transferability, likely due to the multi-layered detection mechanisms employed in commercial systems. Notably, adversarial samples targeting Avast achieved an ASR of up to 67% on VirusTotal. Moreover, Flash outperformed MalwareTotal, indicating that adversarial samples generated by Flash possess stronger transferability.

Robustness Enhancement. To investigate how Flash can strengthen existing detectors, we adopt adversarial training [45, 64] as a defense strategy. Specifically, we generate approximately 1,500 adversarial samples against the MalConvGCT model using Flash. Following the fine-tuning methodology employed in MalwareTotal, we retrain MalConv with these adversarial examples. Subsequently, we evaluate the robustness of the fine-tuned model on the remaining 500 PE files in the test set under several effective black-box attacks, including Gamma Section (GS), Gamma Padding (GP), MAB, MalwareTotal, and Flash itself. As shown in Figure 5(c), the ASR of Flash drops by 82.4% after fine-tuning, and the ASRs of other previously unseen attacks also decrease significantly. We attribute this improvement to Flash's diverse modification patterns, which likely

overlap with the transformation paradigms used by other attacks, enabling adversarial training with Flash to effectively mitigate similar threats.

Answer to RQ4: Flash-generated attacks exhibit strong transferability across commercial AVs, achieving up to 67% ASR on VirusTotal. Furthermore, adversarial training with Flash significantly improves detector robustness, reducing Flash's ASR by 82.4% and mitigating other unseen attacks.

7 Discussion

Generalization Beyond PE Format. While our work primarily focuses on adversarial malware generation within the PE format, the proposed Flash framework is not inherently limited to this file type. The two-stage architecture, which integrates a GAN-based generator with an evolutionary search strategy, is designed to be format-agnostic. The only format-specific component lies in the set of transformation primitives, which, in our case, are tailored to PE files (e.g., header modifications, section injections, or metadata alterations). To extend Flash to other platforms such as Android APKs or ELF executables [39], one would need to define a new collection of semantic-preserving manipulations specific to the target format. We envision this as an important avenue for future work, as it would not only validate the generalizability of Flash across heterogeneous malware ecosystems but also strengthen its practical value in real-world adversarial defense and robustness evaluations.

Limitations. Although Flash significantly reduces the number of queries by combining query-based and transfer-based attacks, the risk of detection still exists in real-world deployments. For example, some defensive mechanisms designed to resist query-based adversarial attacks deploy multi-query detection strategies on the server side [41, 42, 52]. These methods analyze the similarity between consecutive adversarial samples to determine whether the attacker is probing the model to optimize the perturbation sequence. Nevertheless, Flash can still be integrated with more advanced techniques to evade such defenses [10, 22]. For instance, attackers can fine-tune the attack algorithm during the attack process to reduce the similarity between new queries and historical ones, thereby mitigating detection risks [22], while the study of black-box attack queries and corresponding defenses remains an ongoing research topic, which we leave for future work.

Defensive Methods. Although we propose Flash from the perspective of generating functional adversarial samples to attack malware detectors, our ultimate goal is to identify the shortcomings of current malware detection systems and commercial antivirus solutions in order to facilitate their improvement and better defend against more advanced malware threats. To this end, we also conduct adversarial training in Section 6.5 to enhance the robustness of existing detectors and improve their detection performance. In addition, static detectors typically serve as the first line of defense against malware [28], although some stealthy malware samples may bypass static analysis, they can often be detected by subsequent API call sequences [13], I/O behaviors [71] or dynamic malware detection techniques [9, 27]. Finally, we note that several recent advances have enabled the development of more sophisticated detection models [6, 69], which exhibit increased resistance to adversarial perturbations and thus lay the foundation for more robust malware detection in the future.

8 Conclusion

We present Flash, a scalable black-box attack framework for PE malware detection that unifies transfer and query strategies. Flash employs a multi-discriminator GAN and evolutionary optimization to craft valid adversarial PE samples, cutting queries by 86% while achieving up to 100%

evasion. Beyond exposing vulnerabilities, Flash also strengthens detectors via adversarial training, reducing ASR by 82.4% and improving robustness against unseen attacks.

Acknowledgments

This research was supported in part by the National Natural Science Foundation of China (625B2139, 62572377, 62302362) and Shaanxi Province Natural Science Basic Research Program (Project No. 2025SYS-SYSZD-081)

9 Data Availability

In line with open science principles, our complete source code has been made available in the <https://github.com/anyuan1999/Flash.git> to ensure the reproducibility of our methods and results.

References

- [1] Hyrum S Anderson, Anant Kharkar, Bobby Filar, David Evans, and Phil Roth. 2018. Learning to evade static PE machine learning malware models via reinforcement learning. *arXiv preprint arXiv:1801.08917* (2018). doi:10.48550/arXiv.1801.08917
- [2] Hyrum S Anderson, Anant Kharkar, Bobby Filar, and Phil Roth. 2017. Evading machine learning malware detection. *black Hat 2017* (2017), 1–6. <https://api.semanticscholar.org/CorpusID:26406954>
- [3] Hyrum S Anderson and Phil Roth. 2018. Ember: an open dataset for training static pe malware machine learning models. *arXiv preprint arXiv:1804.04637* (2018). doi:10.48550/arXiv.1804.04637
- [4] Avast Software. 2025. Avast: Free antivirus is your first step to online freedom. <https://www.avast.com/en-in>. Accessed: 2025-05-20.
- [5] Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, and Anis Zouaoui. 2024. A survey on malware detection with graph representation learning. *Comput. Surveys* 56, 11 (2024), 1–36. doi:10.1145/3664649
- [6] Hamid Bostani, Zhengyu Zhao, Zhuoran Liu, and Veelasha Moonsamy. 2025. Level up with ML vulnerability identification: leveraging domain constraints in feature space for robust android malware detection. *ACM Transactions on Privacy and Security* 28, 2 (2025), 1–32. doi:10.1145/3711899
- [7] Ero Carrera. 2025. pefile. <https://github.com/erocarrera/pefile>. Accessed: 2025-05-20.
- [8] Raphael Labaca Castro, Corinna Schmitt, and Gabi Dreo. 2019. Aimed: Evolving malware with genetic programming to evade detection. In *2019 18th IEEE international conference on trust, security and privacy in computing and communications/13th IEEE international conference on big data science and engineering (TrustCom/BigDataSE)*. IEEE, 240–247. doi:10.1109/TrustCom/BigDataSE.2019.00040
- [9] Yuhang Chai, Lei Du, Jing Qiu, Lihua Yin, and Zhihong Tian. 2022. Dynamic prototype network based on sample adaptation for few-shot malware detection. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4754–4766. doi:10.1109/TKDE.2022.3142820
- [10] Steven Chen, Nicholas Carlini, and David Wagner. 2020. Stateful detection of black-box adversarial attacks. In *Proceedings of the 1st ACM Workshop on Security and Privacy on Artificial Intelligence*. 30–39. doi:10.1145/3385003.3410925
- [11] Cisco. 2025. ClamAV. <https://www.clamav.net/>. Accessed: 2025-05-20.
- [12] Corvus Forensics. 2025. VirusShare.com - Because Sharing is Caring. <https://virusshare.com/>. Accessed: 2025-05-20.
- [13] Lei Cui, Jiancong Cui, Yuede Ji, Zhiyu Hao, Lun Li, and Zhenquan Ding. 2023. Api2vec: Learning representations of api sequences for malware detection. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 261–273. doi:10.1145/3597926.3598054
- [14] Hung Dang, Yue Huang, and Ee-Chien Chang. 2017. Evading classifiers by morphing in the dark. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 119–133. doi:10.1145/3133956.3133978
- [15] Luca Demetrio and Battista Biggio. 2021. secml-malware: A Python Library for Adversarial Robustness Evaluation of Windows Malware Classifiers. *arXiv:2104.12848 [cs.CR]* doi:10.48550/arXiv.2104.12848
- [16] Luca Demetrio, Battista Biggio, Giovanni Lagorio, Fabio Roli, and Alessandro Armando. 2019. Explaining vulnerabilities of deep learning to adversarial malware binaries. *arXiv preprint arXiv:1901.03583* (2019). doi:10.48550/arXiv.1901.03583
- [17] Luca Demetrio, Battista Biggio, Giovanni Lagorio, Fabio Roli, and Alessandro Armando. 2021. Functionality-preserving black-box optimization of adversarial windows malware. *IEEE Transactions on Information Forensics and Security* 16 (2021), 3469–3478. doi:10.1109/TIFS.2021.3082330
- [18] Luca Demetrio, Scott E Coull, Battista Biggio, Giovanni Lagorio, Alessandro Armando, and Fabio Roli. 2021. Adversarial examples: A survey and experimental evaluation of practical attacks on machine learning for windows malware detection. *ACM Transactions on Privacy and Security (TOPS)* 24, 4 (2021), 1–31. doi:10.1145/3473039

- [19] Ambra Demontis, Marco Melis, Maura Pintor, Matthew Jagielski, Battista Biggio, Alina Oprea, Cristina Nita-Rotaru, and Fabio Roli. 2019. Why do adversarial attacks transfer? explaining transferability of evasion and poisoning attacks. In *28th USENIX security symposium (USENIX security 19)*. 321–338. <https://www.usenix.org/conference/usenixsecurity19/presentation/demontis>
- [20] Yinpeng Dong, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. 2018. Boosting adversarial attacks with momentum. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 9185–9193. doi:10.1109/CVPR.2018.00957
- [21] Mohammadreza Ebrahimi, Jason Pacheco, Weifeng Li, James Lee Hu, and Hsinchun Chen. 2021. Binary black-box attacks against static malware detectors with reinforcement learning in discrete action spaces. In *2021 IEEE security and privacy workshops (SPW)*. IEEE, 85–91. doi:10.1109/SPW53761.2021.00021
- [22] Ryan Feng, Ashish Hooda, Neal Mangaokar, Kassem Fawaz, Somesh Jha, and Atul Prakash. 2023. Stateful defenses for machine learning models are not yet secure against black-box attacks. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 786–800. doi:10.1145/3576915.3623116
- [23] Matthew G Gaber, Mohiuddin Ahmed, and Helge Janicke. 2024. Malware detection with artificial intelligence: A systematic literature review. *Comput. Surveys* 56, 6 (2024), 1–33. doi:10.1145/3638552
- [24] Cuiying Gao, Gaozhun Huang, Heng Li, Bang Wu, Yueming Wu, and Wei Yuan. 2024. A comprehensive study of learning-based android malware detectors under challenging environments. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3623320
- [25] Jiaxuan Geng, Junfeng Wang, Zhiyang Fang, Yingjie Zhou, Di Wu, and Wenhan Ge. 2024. A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack. *Computers & Security* 137 (2024), 103595. doi:10.1016/j.cose.2023.103595
- [26] Daniel Gibert, Jordi Planes, Quan Le, and Giulio Zizzo. 2023. A wolf in sheep’s clothing: Query-free evasion attacks against machine learning-based malware detectors with generative adversarial networks. In *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 415–426. doi:10.1109/EuroSPW59978.2023.00052
- [27] Sibel Gulmez, Arzu Gorgulu Kakisim, and Ibrahim Sogukpinar. 2024. XRank: Explainable deep learning-based ransomware detection using dynamic analysis. *Computers & Security* 139 (2024), 103703. doi:10.1016/j.cose.2024.103703
- [28] Jingcai Guo, Song Guo, Shiheng Ma, Yuxia Sun, and Yuanyuan Xu. 2021. Conservative novelty synthesizing network for malware recognition in an open-set scenario. *IEEE Transactions on Neural Networks and Learning Systems* 34, 2 (2021), 662–676. doi:10.1109/TNNLS.2021.3099122
- [29] Richard Harang and Ethan M Rudd. 2020. SOREL-20M: A large scale benchmark dataset for malicious PE detection. *arXiv preprint arXiv:2012.07634* (2020). doi:10.48550/arXiv.2012.07634
- [30] Shuai He, Cai Fu, Hong Hu, Jiahe Chen, Jianqiang Lv, and Shuai Jiang. 2024. MalwareTotal: Multi-faceted and sequence-aware bypass tactics against static malware detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639141
- [31] Weiwei Hu and Ying Tan. 2022. Generating adversarial malware examples for black-box attacks based on GAN. In *International Conference on Data Mining and Big Data*. Springer, 409–423. doi:10.1007/978-981-19-8991-9_29
- [32] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144* (2016). doi:10.48550/arXiv.1611.01144
- [33] Robert J. Joyce, Gideon Miller, Phil Roth, Richard Zak, Elliott Zaresky-Williams, Hyrum Anderson, Edward Raff, and James Holt. 2025. EMBER2024 - A Benchmark Dataset for Holistic Evaluation of Malware Classifiers. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. doi:10.1145/3711896.3737431
- [34] Temesguen Messay Kebede, Ouboti Djaneye-Boundjou, Barath Narayanan Narayanan, Anca Ralescu, and David Kapp. 2017. Classification of malware programs using autoencoders based deep learning architecture and its application to the microsoft malware classification challenge (big 2015) dataset. In *2017 IEEE National Aerospace and Electronics Conference (NAECON)*. IEEE, 70–75. doi:10.1109/NAECON.2017.8268747
- [35] Bojan Kolosnjaji, Ambra Demontis, Battista Biggio, Davide Maiorca, Giorgio Giacinto, Claudia Eckert, and Fabio Roli. 2018. Adversarial malware binaries: Evading deep learning for malware detection in executables. In *2018 26th European signal processing conference (EUSIPCO)*. IEEE, 533–537. doi:10.23919/EUSIPCO.2018.8553214
- [36] Bojan Kolosnjaji, Apostolis Zarras, George Webster, and Claudia Eckert. 2016. Deep learning for classification of malware system call sequences. In *AI 2016: Advances in Artificial Intelligence: 29th Australasian Joint Conference, Hobart, TAS, Australia, December 5-8, 2016, Proceedings 29*. Springer, 137–149. doi:10.1007/978-3-319-50127-7_11
- [37] Felix Kreuk, Assi Barak, Shir Aviv-Reuven, Moran Baruch, Benny Pinkas, and Joseph Keshet. 2018. Adversarial examples on discrete sequences for beating whole-binary malware detection. *arXiv preprint arXiv:1802.04528* (2018), 490–510. doi:10.48550/arXiv.1802.04528
- [38] Raphael Labaca-Castro, Sebastian Franz, and Gabi Dreo Rodosek. 2021. AIMED-RL: Exploring adversarial malware examples with reinforcement learning. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part IV 21*.

- Springer, 37–52. doi:10.1007/978-3-030-86514-6_3
- [39] Deqiang Li and Qianmu Li. 2020. Adversarial deep ensemble: Evasion attacks and defenses for malware detection. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3886–3900. doi:10.1109/TIFS.2020.3003571
- [40] Deqiang Li, Qianmu Li, Yanfang Ye, and Shouhuai Xu. 2021. Arms race in adversarial malware detection: A survey. *ACM Computing Surveys (CSUR)* 55, 1 (2021), 1–35. doi:10.1145/3484491
- [41] Huiying Li, Shawn Shan, Emily Wenger, Jiayun Zhang, Haitao Zheng, and Ben Y Zhao. 2022. Blacklight: Scalable defense for neural networks against {Query-Based}{Black-Box} attacks. In *31st USENIX Security Symposium (USENIX Security 22)*. 2117–2134. <https://www.usenix.org/conference/usenixsecurity22/presentation/li-huiying>
- [42] Shaofei Li, Ziqi Zhang, Haomin Jia, Yao Guo, Xiangqun Chen, and Ding Li. 2025. Query Provenance Analysis: Efficient and Robust Defense against Query-based Black-box Attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1641–1656. doi:10.1109/SP61157.2025.00072
- [43] Xintong Li and Qi Li. 2021. An IRL-based malware adversarial generation method to evade anti-malware engines. *Computers & Security* 104 (2021), 102118. doi:10.1016/j.cose.2020.102118
- [44] Xiang Ling, Lingfei Wu, Jiangyu Zhang, Zhenqing Qu, Wei Deng, Xiang Chen, Yaguan Qian, Chunming Wu, Shouling Ji, Tianyue Luo, et al. 2023. Adversarial attacks against Windows PE malware detection: A survey of the state-of-the-art. *Computers & Security* 128 (2023), 103134. doi:10.1016/j.cose.2023.103134
- [45] Keane Lucas, Samruddhi Pai, Weiran Lin, Lujo Bauer, Michael K Reiter, and Mahmood Sharif. 2023. Adversarial training for {Raw-Binary} malware classifiers. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1163–1180. <https://www.usenix.org/conference/usenixsecurity23/presentation/lucas>
- [46] Keane Lucas, Mahmood Sharif, Lujo Bauer, Michael K Reiter, and Saurabh Shintre. 2021. Malware makeover: Breaking ml-based static analysis by modifying executable bytes. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*. 744–758. doi:10.1145/3433210.3453086
- [47] Zhaoyi Meng, Jiale Zhang, Jiaqi Guo, Wansen Wang, Wenchao Huang, Jie Cui, Hong Zhong, and Yan Xiong. 2025. Detecting Android Malware by Visualizing App Behaviors From Multiple Complementary Views. *IEEE Transactions on Information Forensics and Security* (2025). doi:10.1109/TIFS.2025.3547301
- [48] Vitaly Morgunov. 2023. IoT-Threat-Report. <https://securelist.com/iot-threat-report-2023/110644/>. Accessed: 2025-05-20.
- [49] Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. 2020. Intriguing Properties of Adversarial ML Attacks in the Problem Space. In *Proc. 2020 IEEE Symposium on Security and Privacy*. IEEE, 1332–1349. doi:10.1109/SP40000.2020.00073
- [50] Edward Raff, Jon Barker, Jared Sylvester, Robert Brandon, Bryan Catanzaro, and Charles K Nicholas. 2018. Malware detection by eating a whole exe. In *Workshops at the thirty-second AAAI conference on artificial intelligence*.
- [51] Edward Raff, William Fleschman, Richard Zak, Hyrum Anderson, Bobby Filar, and Mark Mclean. 2021. Classifying Sequences of Extreme Length with Constant Memory Applied to Malware Detection. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence*. doi:10.1609/aaai.v35i11.17131
- [52] Aqib Rashid and Jose Such. 2023. Malprotect: Stateful defense against adversarial query attacks in ml-based malware detection. *IEEE Transactions on Information Forensics and Security* 18 (2023), 4361–4376. doi:10.1109/TIFS.2023.3293959
- [53] Ishai Rosenberg, Asaf Shabtai, Yuval Elovici, and Lior Rokach. 2020. Query-efficient black-box attack against sequence-based malware classifiers. In *Proceedings of the 36th Annual Computer Security Applications Conference*. 611–626. doi:10.1145/3427228.3427230
- [54] Anyuan Sang, Zhipeng Wang, Li Yang, Lu Zhou, Junbo Jia, and Huipeng Yang. 2025. Generating Adversarial Malware Examples Against Multiple Machine Learning Detectors. *IEEE Transactions on Industrial Informatics* (2025), 1–11. doi:10.1109/TII.2025.3556075
- [55] Matthew G Schultz, Eleazar Eskin, F Zadok, and Salvatore J Stolfo. 2000. Data mining methods for detection of new malicious executables. In *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. IEEE, 38–49. doi:10.1109/SECPRI.2001.924286
- [56] Wei Song, Xuezixiang Li, Sadia Afroz, Deepali Garg, Dmitry Kuznetsov, and Heng Yin. 2022. MAB-Malware: A Reinforcement Learning Framework for Blackbox Generation of Adversarial Malware. In *Proc. 2022 ACM on Asia Conf. on Computer and Communications Security*. 990–1003. doi:10.1145/3488932.3497768
- [57] Octavian Suci, Scott E Coull, and Jeffrey Johns. 2019. Exploring adversarial examples in malware detection. In *2019 IEEE Security and Privacy Workshops*. IEEE, 8–14. doi:10.1109/SPW.2019.00015
- [58] Romain Thomas. 2020. A Command line tool for launching attacks against Machine Learning Malware detectors. <https://github.com/pralab/toucanstrike/>. Accessed: 2025-05-20.
- [59] Buwei Tian, Junyong Jiang, Zichen He, Xin Yuan, Lu Dong, and Changyin Sun. 2024. Functionality-verification attack framework based on reinforcement learning against static malware detectors. *IEEE Transactions on Information Forensics and Security* (2024). doi:10.1109/TIFS.2024.3453047
- [60] VirusTotal. 2025. VirusTotal. <https://www.virustotal.com/gui/home/upload>. Accessed: 2025-05-20.

- [61] Jialai Wang, Wenjie Qu, Yi Rong, Han Qiu, Qi Li, Zongpeng Li, and Chao Zhang. 2023. MPass: Bypassing Learning-based Static Malware Detectors. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6. doi:10.1109/DAC56929.2023.10247858
- [62] Wei Wang, Ruoxi Sun, Tian Dong, Shaofeng Li, Minhui Xue, Gareth Tyson, and Haojin Zhu. 2021. Exposing weaknesses of malware detectors with explainability-guided evasion attacks. *arXiv e-prints* (2021), arXiv–2111. doi:10.48550/arXiv.2111.10085
- [63] Cihang Xie, Zhishuai Zhang, Yuyin Zhou, Song Bai, Jianyu Wang, Zhou Ren, and Alan L Yuille. 2019. Improving transferability of adversarial examples with input diversity. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2730–2739. doi:10.1109/CVPR.2019.00284
- [64] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural attack for pre-trained models of code. In *Proceedings of the 44th International Conference on Software Engineering*. 1482–1493. doi:10.1145/3510003.3510146
- [65] Yuan Junkun, Zhou Shaofang, Lin Lanfen, Wang Feng, and Cui Jia. 2020. Black-box adversarial attacks against deep learning based malware binaries detection with GAN. In *ECAI 2020*. IOS Press, 2536–2542. doi:10.3233/FAIA200388
- [66] Javier Yuste, Eduardo G Pardo, and Juan Tapiador. 2022. Optimization of code caves in malware binaries to evade machine learning detectors. *Computers & Security* 116 (2022), 102643. doi:10.1016/j.cose.2022.102643
- [67] Dazhi Zhan, Yexin Duan, Yue Hu, Lujia Yin, Zhisong Pan, and Shize Guo. 2023. AMGmal: Adaptive mask-guided adversarial attack against malware detection with minimal perturbation. *Computers & Security* 127 (2023), 103103. doi:10.1016/j.cose.2023.103103
- [68] Dazhi Zhan, Kun Xu, Xin Liu, Tong Han, Zhisong Pan, and Shize Guo. 2025. Practical clean-label backdoor attack against static malware detection. *Computers & Security* 150 (2025), 104280. doi:10.1016/j.cose.2024.104280
- [69] Jingnan Zheng, Jiahao Liu, An Zhang, Jun Zeng, Ziqi Yang, Zhenkai Liang, and Tat-Seng Chua. 2024. MaskDroid: Robust Android Malware Detection with Masked Graph Representations. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 331–343. doi:10.1145/3691620.3695008
- [70] Fangtian Zhong, Xiuzhen Cheng, Dongxiao Yu, Bei Gong, Shuaiwen Song, and Jiguo Yu. 2023. MalFox: Camouflaged adversarial malware example generation based on conv-GANs against black-box detectors. *IEEE Trans. Comput.* 73, 4 (2023), 980–993. doi:10.1109/TC.2023.3236901
- [71] Chijin Zhou, Lihua Guo, Yiwei Hou, Zhenya Ma, Quan Zhang, Mingzhe Wang, Zhe Liu, and Yu Jiang. 2023. Limits of i/o based ransomware detection: An imitation based attack. In *2023 IEEE symposium on security and privacy (SP)*. IEEE, 2584–2601. doi:10.1109/SP46215.2023.10179372

Received 2025-09-02; accepted 2025-12-22